

React Native Interview Questions — Quick Revision (Most Important, Covers Everything)

Short version covering the full breadth of React Native — fundamentals to deployment. One condensed question per topic. Functional components + hooks + TypeScript throughout.

Table of Contents

Fundamentals: [1. RN Architecture](#) · [2. Hermes](#) · [3. Functional vs Class](#) · [4. Props vs State](#)

Hooks: [5. useState](#) · [6. useEffect](#) · [7. useRef](#) · [8. useMemo/useCallback](#) · [9. useReducer](#) · [10. Custom Hooks](#) · [11. useFocusEffect](#)

Navigation: [12. Stack/Tab/Drawer](#) · [13. Deep Linking](#)

State & API: [14. Redux Toolkit](#) · [15. RTK Query](#) · [16. Context API](#) · [17. Axios](#)

Storage & Lists: [18. AsyncStorage vs Secure Storage](#) · [19. FlatList & Optimization](#)

Performance: [20. React.memo, useMemo, useCallback](#)

Native & Firebase: [21. Native Modules/Bridge](#) · [22. Firebase Push Notifications](#) · [23. Permissions](#)

Libraries, Styling, Testing, Release: [24. Key Libraries](#) · [25. Flexbox](#) · [26. Testing](#) · [27. Build & Release](#) · [28. CI/CD](#) · [29. Security](#)

1. React Native Architecture (Bridge, New Architecture, Fabric, TurboModules, JSI)

Answer

Old architecture: JS and native code talk through the **Bridge**, converting data to JSON asynchronously — slow for heavy/frequent calls. New architecture uses **JSI** to let JS call native code directly and synchronously. **Fabric** is the new renderer (faster UI updates), **TurboModules** load native modules only when needed (faster app start).

Example

```
Old: JS <--JSON over async Bridge--> Native
New: JS <--JSI, direct sync calls--> Native
(Fabric + TurboModules)
```

Real-world Example

Apps with heavy native work (camera, real-time animations) feel noticeably smoother on the New Architecture.

Follow-up Questions

- Why was the Bridge a bottleneck?
 - What problem does JSI solve?
-

2. Hermes Engine

Answer

Hermes is Meta's JS engine built for React Native — gives faster app startup, smaller app size, lower memory, by pre-compiling JS to bytecode instead of parsing raw JS at launch.

Example

```
// Hermes is enabled by default in modern
RN; historically toggled via:
// android/app/build.gradle ->
enableHermes: true
```

Real-world Example

Easiest performance win for cold start time — most teams just enable it, no app code changes needed.

Follow-up Questions

- How does Hermes speed up startup?
 - Is it on by default today?
-

3. Functional vs Class Components

Answer

Class components use `this`, state, and lifecycle methods. Functional components use hooks (`useState`, `useEffect`). Modern RN apps use functional components almost exclusively — shorter, no `this` bugs.

Example

```
function Counter() {  
  const [count, setCount] = useState(0);  
  return <Button title={`${count}`}   
    onPress={() => setCount(count + 1)} />;  
}
```

Real-world Example

Nearly every new screen today is functional; class components mostly appear in legacy code.

Follow-up Questions

- Why did React move to hooks?
 - Hook equivalent of `componentDidMount` ?
-

4. Props vs State

Answer

Props come from the parent and are read-only. State lives inside the component and can change, triggering a re-render.

Example

```
function Greeting({ name }: { name: string }) { return <Text>Hi {name}</Text>; } // prop
const [liked, setLiked] = useState(false);
// state
```

Real-world Example

Product data in a FlatList item comes as props; whether "Add to Cart" is loading is local state.

Follow-up Questions

- Can a child change its own props?
 - How does data flow from child back to parent?
-

5. useState

Answer

Lets a component hold and update its own data.

Returns `[value, setterFunction]`.

Example

```
const [count, setCount] = useState(0);
setCount((prev) => prev + 1); // safer when
depending on old value
```

Real-world Example

Used for text input values, modal visibility, loading flags.

Follow-up Questions

- Why use `setCount(prev => prev+1)` over `setCount(count+1)` ?
- Is `setState` sync or async?

6. useEffect

Answer

Runs side effects (API calls, subscriptions) after render. `[]` = run once on mount, `[value]` = run when value changes, no array = every render. Return a cleanup function to avoid leaks.

Example

```
useEffect(() => {  
  let active = true;  
  axios.get(`/users/${id}`).then((r) =>  
    active && setUser(r.data));  
  return () => { active = false; };  
}, [id]);
```

Real-world Example

Fetching data on screen load;
subscribing/unsubscribing to a Firebase listener.

Follow-up Questions

- Difference between `[]` and no dependency array?
- Why does the cleanup function matter?

7. useRef

Answer

Holds a value across renders without causing re-renders. Used for referencing native elements (focus a TextInput) or storing mutable values like timer IDs.

Example

```
const inputRef = useRef<TextInput>(null);
<TextInput ref={inputRef} />;
inputRef.current?.focus();
```

Real-world Example

Auto-focusing the next field in an OTP screen.

Follow-up Questions

- Why doesn't changing `.current` re-render?
- `useRef` vs `useState` — when to use which?

8. useMemo vs useCallback

Answer

`useMemo` caches a **calculated value**, recalculating only when dependencies change. `useCallback` caches a **function reference** so it doesn't get recreated every render — useful with `React.memo` children.

Example

```
const filtered = useMemo(() =>
  products.filter(p => p.name.includes(q)),
  [products, q]);
const handlePress = useCallback(() =>
  doSomething(), []);
```

Real-world Example

`useMemo` for filtering a 1000+ item product list;
`useCallback` for `onPress` passed into memoized `FlatList` items.

Follow-up Questions

- What problem does `useCallback` solve with `React.memo` ?
- Can overusing these hurt performance?

9. `useReducer`

Answer

Alternative to `useState` for complex state logic — dispatch an action, a reducer function decides the new state. Works like a mini-Redux.

Example

```
function reducer(state, action) {
  switch (action.type) {
    case "increment": return { count:
state.count + 1 };
    default: return state;
  }
}
const [state, dispatch] =
useReducer(reducer, { count: 0 });
```

Real-world Example

Managing a multi-step checkout flow or complex form with many related fields.

Follow-up Questions

- When choose `useReducer` over `useState` ?
- How does it relate to Redux?

10. Custom Hooks

Answer

A function starting with `use` that calls other hooks, used to reuse logic across components instead of copy-pasting.

Example

```
function useDebounce<T>(value: T, delay: number): T {
  const [debounced, setDebounced] = useState(value);
  useEffect(() => {
    const t = setTimeout(() =>
      setDebounced(value), delay);
    return () => clearTimeout(t);
  }, [value, delay]);
  return debounced;
}
```

Real-world Example

`useDebounce`, `useAuth`, `useNetworkStatus`, `useApi` are common in real apps.

Follow-up Questions

- What are the rules of hooks?
 - Can a custom hook call another custom hook?
-

11. `useFocusEffect`

Answer

From React Navigation — runs an effect every time a screen comes into **focus**, not just on first mount. Needed because screens stay mounted in the background in a stack navigator.

Example

```
useFocusEffect(  
  useCallback(() => {  
    fetchLatestProfile();  
  }, [])  
);
```

Real-world Example

Refreshing profile data when navigating back to that screen after editing it elsewhere.

Follow-up Questions

- Why not just use `useEffect` here?
- Difference from `useIsFocused` ?

12. Stack, Bottom Tab, and Drawer Navigation

Answer

Stack pushes/pops screens like a stack of cards (back button works naturally). **Bottom Tabs** show persistent tabs at the bottom for switching between top-level sections. **Drawer** is a slide-in side menu. All three are usually combined – e.g. tabs inside a stack.

Example

```
const Stack = createNativeStackNavigator();
<Stack.Navigator>
  <Stack.Screen name="Home" component=
{HomeScreen} />
  <Stack.Screen name="Details" component=
{DetailsScreen} />
</Stack.Navigator>
```

Real-world Example

E-commerce app: Bottom Tabs for Home/Cart/Profile, each tab containing its own Stack for drill-down screens.

Follow-up Questions

- How do you pass params between screens?
 - How do nested navigators work?
-

13. Deep Linking

Answer

Lets the app open directly to a specific screen from a URL (e.g. from a notification, email, or another app), instead of always opening the home screen.

Example

```
const linking = {
  prefixes: ["myapp://"],
  config: { screens: { Details:
    "product/:id" } },
};
<NavigationContainer linking={linking}>...
</NavigationContainer>
```

Real-world Example

Tapping a "View Order" push notification opens the Order Details screen directly.

Follow-up Questions

- How is deep linking configured for both Android and iOS?
 - How does deep linking work with Firebase dynamic links?
-

14. Redux Toolkit (createSlice, configureStore, useSelector, useDispatch)

Answer

Redux Toolkit is the modern, recommended way to use Redux. `createSlice` generates action creators and a reducer together. `configureStore` sets up the store with good defaults. `useSelector` reads state, `useDispatch` sends actions – both used inside components.

Example

```
const counterSlice = createSlice({
  name: "counter",
  initialState: { value: 0 },
  reducers: { increment: (state) => {
    state.value += 1; } }, // Immer allows
    "mutation"
  });
const store = configureStore({ reducer: {
  counter: counterSlice.reducer } });

// In component
const count = useSelector((state:
RootState) => state.counter.value);
const dispatch = useDispatch();
dispatch(counterSlice.actions.increment());
```

Real-world Example

Global auth state (logged-in user), cart state, theme settings shared across many screens.

Follow-up Questions

- How does Immer let you "mutate" state inside a slice safely?
 - Why is RTK preferred over old plain Redux?
-

15. RTK Query

Answer

A data-fetching tool built into Redux Toolkit. It auto-generates hooks for API calls and handles caching, loading/error states, and refetching — removing the need to manually write thunks/reducers for API calls.

Example

```
const api = createApi({
  baseQuery: fetchBaseQuery({ baseUrl:
    "/api" }),
  endpoints: (builder) => ({
    getUser: builder.query<User, number>({
      query: (id) => `/users/${id}`,
    }),
  }),
});
```

```
});  
export const { useGetUserQuery } = api;  
  
// In component  
const { data, isLoading, error } =  
  useGetUserQuery(1);
```

Real-world Example

Replaces manually writing `useEffect` + `axios` + loading/error state for every API call in the app.

Follow-up Questions

- How does RTK Query handle caching automatically?
- RTK Query vs writing Axios calls manually — when would you choose each?

16. Context API

Answer

Built into React — lets you share data across many components without manually passing props through every level ("prop drilling"). Best for low-frequency-change global data like theme or auth user.

Example

```
const AuthContext = createContext<{ user:
User | null }>({ user: null });
function App() {
  return <AuthContext.Provider value={{
user }}><HomeScreen />
</AuthContext.Provider>;
}
const { user } = useContext(AuthContext);
```

Real-world Example

Theme (dark/light mode) or logged-in user info available across the whole app.

Follow-up Questions

- When would you choose Redux over Context API?
 - Does Context cause unnecessary re-renders? How to avoid it?
-

17. Axios (GET, POST, PUT, DELETE, Interceptors)

Answer

Axios is a Promise-based HTTP client used for API calls. Supports all HTTP methods, and **interceptors** let you run code before every request (e.g. attach auth

token) or after every response (e.g. handle 401 globally).

Example

```
const api = axios.create({ baseURL:
  "https://api.example.com" });

api.interceptors.request.use((config) => {
  config.headers.Authorization = `Bearer
  ${token}`;
  return config;
});

api.interceptors.response.use(
  (res) => res,
  (error) => {
    if (error.response?.status === 401)
      logoutUser();
    return Promise.reject(error);
  }
);

await api.get("/users");
await api.post("/users", { name: "Asha" });
```

Real-world Example

Attaching a JWT token to every request automatically, and force-logging out the user on a 401 globally, without repeating that logic in every API call.

Follow-up Questions

- Why use interceptors instead of adding headers manually in every call?
 - How would you retry a failed request using interceptors?
-

18. AsyncStorage vs Secure Storage

Answer

`AsyncStorage` stores simple key-value data **unencrypted** on the device — fine for non-sensitive data like theme preference or onboarding-seen flag. For sensitive data (tokens, passwords), use **Secure Storage** (e.g. `react-native-keychain` or `expo-secure-store`), which encrypts data using the device's secure storage (Keychain on iOS, Keystore on Android).

Example

```
await AsyncStorage.setItem("theme",
  "dark");
const theme = await
  AsyncStorage.getItem("theme");

await
```

```
Keychain.setGenericPassword("authToken",  
token); // secure
```

Real-world Example

Storing the auth token in Keychain/Keystore instead of AsyncStorage, since AsyncStorage data isn't encrypted and could be read if the device is compromised.

Follow-up Questions

- Why shouldn't you store auth tokens in plain AsyncStorage?
- Is AsyncStorage synchronous or asynchronous?

19. FlatList & Optimization

Answer

`FlatList` efficiently renders large lists by only rendering items currently visible on screen (virtualization), unlike `ScrollView` which renders everything at once. Key optimization props: `keyExtractor` (stable keys), `getItemLayout` (skip measuring, if item size is fixed), `removeClippedSubviews`, `windowSize`,

`initialNumToRender` , and wrapping `renderItem` components with `React.memo` .

Example

```
<FlatList
  data={products}
  keyExtractor={(item) =>
    item.id.toString()}
  renderItem={({ item }) => <ProductCard
    product={item} />}
  initialNumToRender={10}
  windowSize={5}
  getItemLayout={(data, index) => ({
    length: 80, offset: 80 * index, index })}
/>

const ProductCard = React.memo(({ product
}: { product: Product }) => <Text>
  {product.name}</Text>);
```

Real-world Example

A product catalog with thousands of items stays smooth because FlatList only mounts what's visible plus a small buffer.

Follow-up Questions

- Why is `ScrollView` bad for long lists?
- What does `getItemLayout` actually optimize?

20. Performance Optimization (React.memo, useMemo, useCallback, Lazy Loading)

Answer

`React.memo` skips re-rendering a component if its props haven't changed. `useMemo` / `useCallback` avoid recalculating values/recreating functions unnecessarily. Lazy loading (`React.lazy` / dynamic imports) loads screens/components only when needed, reducing initial bundle size. Image optimization (correct size, caching, `resizeMode`) avoids memory bloat.

Example

```
const ProductCard = React.memo(({ product
}: Props) => <Text>{product.name}</Text>);
const LazyScreen = React.lazy(() =>
import("./HeavyScreen"));
```

Real-world Example

Memoizing `FlatList` item components prevents the whole list from re-rendering when only one item's data changes.

Follow-up Questions

- How do you avoid unnecessary re-renders caused by inline functions/objects as props?
 - How would you profile performance issues in a React Native app?
-

21. Native Modules & Native Bridge

Answer

Native Modules let JavaScript call native Android (Java/Kotlin) or iOS (Swift/Obj-C) code, for things React Native doesn't support out of the box (custom hardware access, specific SDKs). The communication historically goes through the Bridge (or directly via JSI in the New Architecture).

Example

```
import { NativeModules } from "react-native";
const { MyCustomModule } = NativeModules;
MyCustomModule.doSomethingNative();
```

Real-world Example

A custom barcode scanner SDK or a proprietary payment SDK that only exists as a native Android/iOS library gets wrapped in a Native Module.

Follow-up Questions

- When would you need to write a native module instead of using a JS library?
 - How is a TurboModule different from an old-style Native Module?
-

22. Firebase FCM Push Notifications

Answer

Firebase Cloud Messaging (FCM) delivers push notifications. App behavior differs by state:

Foreground (app open) — you usually show a custom in-app alert since the OS won't auto-show a notification. **Background** (app minimized) — OS shows the notification automatically; tapping it can navigate to a specific screen. **Killed** (app fully closed) — a "quit state" handler must be set up separately to handle the notification when the app is opened by tapping it.

Example

```
messaging().onMessage(async (msg) => {
  showInAppAlert(msg); }); // foreground
messaging().setBackgroundMessageHandler(async (msg) => { /* background */ });
```

```
messaging().getInitialNotification().then((  
msg) => { /* killed state */ });
```

Real-world Example

An e-commerce app showing an in-app banner for a flash sale while the user is using the app, vs a system notification when the app is in the background.

Follow-up Questions

- Why is foreground handling different from background?
- How do you navigate to a specific screen from a tapped notification?

23. Permissions (Camera, Location, Storage)

Answer

Sensitive features (camera, location, storage, contacts) require explicit user permission on both Android and iOS. Usually handled with the `react-native-permissions` library or platform-specific APIs, and you should always check/request permission **before** trying to use the feature, and handle the "denied" case gracefully.

Example

```
import { check, request, PERMISSIONS,
RESULTS } from "react-native-permissions";

const result = await
check(PERMISSIONS.IOS.CAMERA);
if (result !== RESULTS.GRANTED) {
  await request(PERMISSIONS.IOS.CAMERA);
}
```

Real-world Example

A QR scanner feature checks camera permission first and shows a friendly message ("Please enable camera access in settings") if denied, instead of crashing.

Follow-up Questions

- What's the difference in how Android and iOS handle permission prompts?
- How do you handle a permanently denied permission (user must go to settings)?

24. Key Libraries Overview

Answer

React Navigation — screen routing/navigation. **Axios** — HTTP client. **Redux Toolkit** — state management. **AsyncStorage** — local key-value storage. **Firebase** — push notifications, analytics, crash reporting. **WebView** — embed web content inside the app. **Lottie** — render After Effects animations. **React Native Maps** — show maps/markers. **Razorpay** (or Stripe) — payment gateway integration.

Example

```
<WebView source={{ uri:
  "https://example.com" }} />
<LottieView source=
  {require("../loading.json")} autoPlay loop
/>
```

Real-world Example

A food delivery app: React Navigation for screens, Redux Toolkit for cart/auth state, Maps for delivery tracking, Razorpay for checkout, Lottie for a loading animation.

Follow-up Questions

- Why would you use a WebView instead of building a native screen?

- What are common gotchas when integrating a payment gateway like Razorpay?
-

25. Flexbox & Responsive Design

Answer

React Native uses Flexbox for layout (similar to CSS, but `flexDirection` defaults to `column`, not `row`, unlike web). Key properties: `flex`, `flexDirection`, `justifyContent` (main axis), `alignItems` (cross axis). For responsive design, use `Dimensions` / `useWindowDimensions` instead of hardcoded pixel values, and percentage-based widths.

Example

```
const { width } = useWindowDimensions();
<View style={{ flex: 1, flexDirection:
"row", justifyContent: "space-between" }}>
  <View style={{ width: width * 0.5 }} />
</View>
```

Real-world Example

Building a card layout that adapts properly between a small phone and a tablet using `useWindowDimensions` instead of fixed widths.

Follow-up Questions

- Difference between `justifyContent` and `alignItems` ?
 - How do you handle different screen sizes/densities across devices?
-

26. Testing (Jest & React Native Testing Library)

Answer

Jest is the test runner/framework used for unit tests (testing pure functions, reducers, utility logic). **React Native Testing Library** is used for testing components – rendering them and simulating user interactions (press, type) the way a real user would, without testing internal implementation details.

Example

```
test("increments count on press", () => {
  const { getByText } = render(<Counter
/>);
  fireEvent.press(getByText("Increment"));
  expect(getByText("1")).toBeTruthy();
});
```

Real-world Example

Testing that a login form shows a validation error when submitted with an empty password field.

Follow-up Questions

- Difference between unit tests and component tests here?
 - How would you mock an Axios API call in a test?
-

27. Build & Release (APK vs AAB)

Answer

APK (Android Package) is the installable file, can be shared directly or used for testing. **AAB** (Android App Bundle) is the format required by the Google Play Store — it lets Google generate optimized APKs per device, resulting in a smaller download size for users. iOS apps are built and submitted as `.ipa` files through Xcode/App Store Connect.

Example

```
# Android release build (AAB for Play Store)
cd android && ./gradlew bundleRelease
```

```
# Android APK (for direct testing/sharing)
cd android && ./gradlew assembleRelease
```

Real-world Example

Uploading an AAB to the Play Store for production release, but generating an APK to quickly share a test build with QA.

Follow-up Questions

- Why does Google Play require AAB now instead of APK?
- What's involved in iOS App Store deployment vs Android?

28. CI/CD (Fastlane & CodePush)

Answer

Fastlane automates building, signing, and uploading apps to the Play Store/App Store, removing manual release steps. **CodePush** (or similar OTA tools) lets you push JS-only updates (bug fixes, small changes) directly to users' devices **without** going through app store review — but native code changes still require a full store release.

Example

```
fastlane android deploy    # automated build
+ upload
appcenter codepush release-react -a
MyOrg/MyApp-Android -d Production
```

Real-world Example

Fixing a critical JS bug (like a broken API call) and pushing it live to all users within minutes via CodePush, instead of waiting days for app store review.

Follow-up Questions

- What kind of changes can't be shipped via CodePush/OTA?
- How does Fastlane fit into a CI pipeline (like GitHub Actions)?

29. Security (Token Storage & SSL Pinning)

Answer

Never store auth tokens in plain AsyncStorage — use encrypted Secure Storage (Keychain/Keystore). **SSL Pinning** adds extra protection by making the app only

trust a specific, known server certificate, instead of any certificate signed by a trusted authority — this prevents man-in-the-middle attacks even if a device's trusted certificate store is compromised.

Example

```
// Token: use Keychain, not AsyncStorage
await Keychain.setGenericPassword("token",
authToken);

// SSL Pinning is typically configured
natively (e.g. via react-native-ssl-pinning
or TrustKit)
```

Real-world Example

A banking app uses SSL pinning so that even on a compromised public Wi-Fi network with a fake certificate, the app refuses to connect.

Follow-up Questions

- Why is SSL pinning important even though HTTPS is already encrypted?
 - What's a downside/maintenance cost of SSL pinning (hint: certificate rotation)?
-

[← Back to top](#)